

Cornell Bowers C-IS

College of Computing and Information Science

Deep Learning

Week [8]: [MDPs/Value]

Varsha Kishore, Justin Lovelace, Max Koster, Yao W, John Denero

Logistics

- HW5 Released this Thursday 04/09 (due 04/23)
- A4 Released this Thursday 04/09 (due 04/21)
- Please check [Ed announcement](#) for Final Project Presentations
 - Last two days of class - Attendance mandatory both days (you either present or grade).

Overview

- Reinforcement Learning
- Markov Decision Processes
- Policy
- Value
- Action-Value

Recap: Supervised Learning

Data: x, y

Data x , label y

Goal: Learn to predict label from data

Examples: Classification, regression,
object detection, semantic
segmentation



→ Cat

Classification

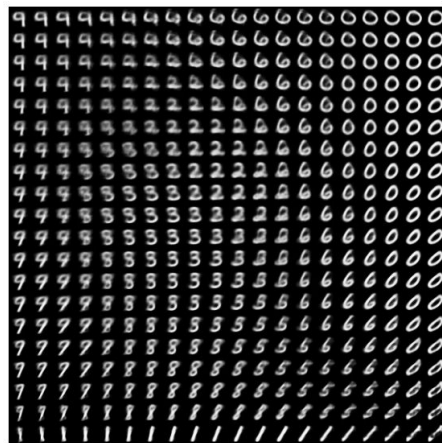
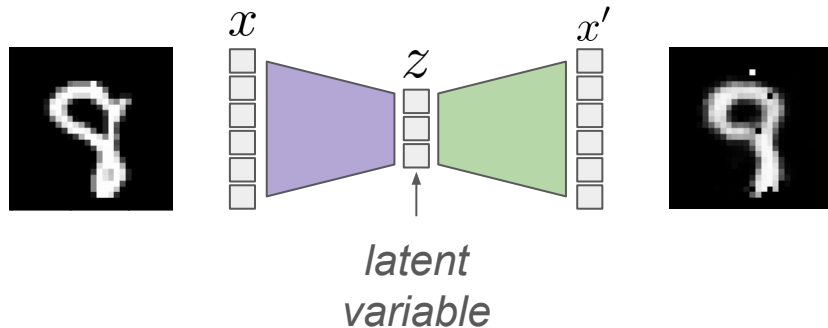
Recap: Unsupervised Learning

Data: x

Just data, no labels!

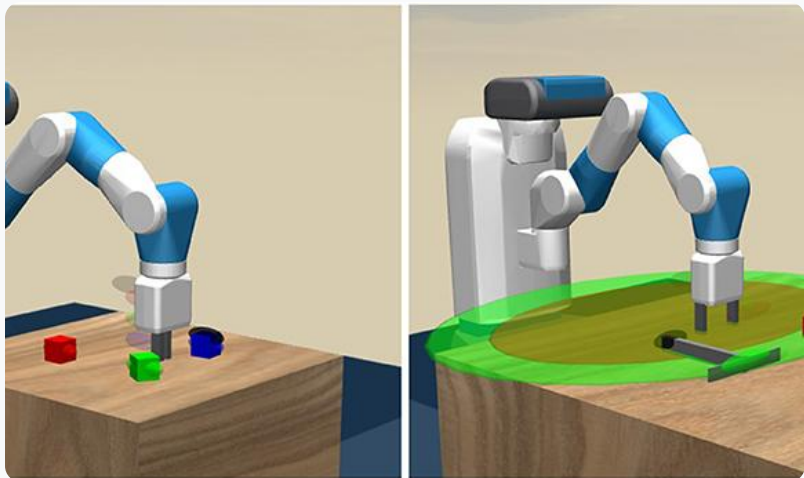
Goal: Learn some underlying hidden *structure* (e.g. manifold / distribution) of the data

Examples: Clustering, dimensionality reduction, generative models (diffusion models, VAEs, GANs), density estimation, Language Models



Limitations of Supervised Learning

Can a regression/classification algorithm learn to perform these tasks successfully?



Robot Learning to Pick Up Blocks

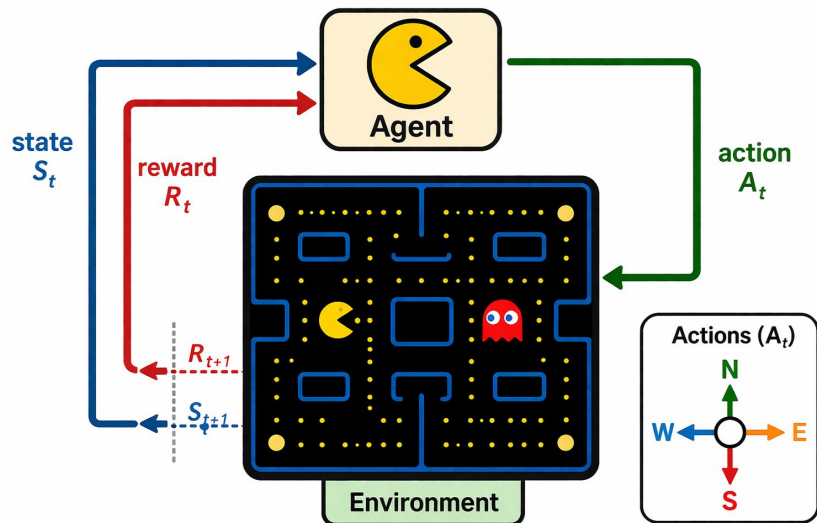


Upside-down helicopter flight

Reinforcement Learning

Problems involving an **agent** interacting with an **environment**, which provides numeric **reward signals**

Goal: Learn how to take actions in order to maximize reward



Agent:

- Perceives environment.
- Makes decisions.
- Aims to maximize reward.

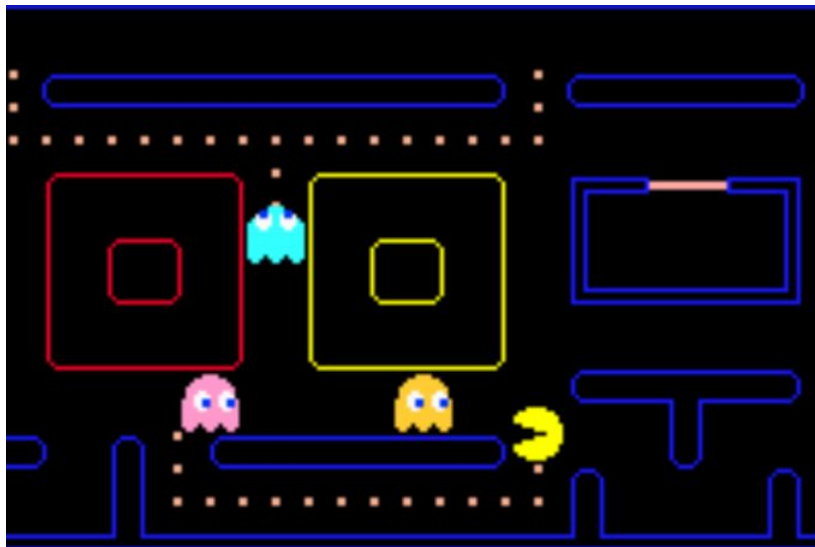
Environment:

- The external context in which an agent operates and interacts with
- Provides feedback to agent

Reward can happen after a long series of actions (e.g. Chess) **Reward Assignment Problem** (i.e. which action caused me to win/lose)

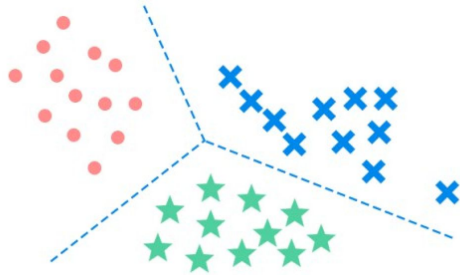
Discussion: Rewards

What are ways to measure rewards in these games?



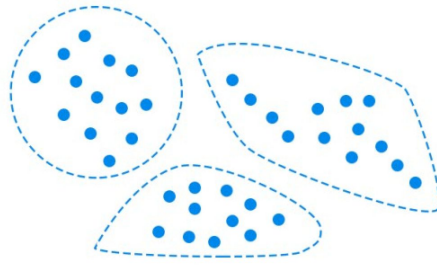
Supervised Learning

- Learns from a labeled dataset
- Classification
Regression



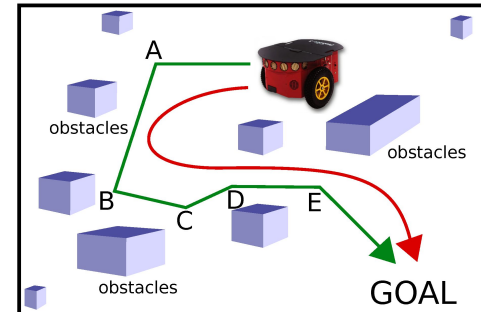
Unsupervised Learning

- Find patterns in unlabeled data
- Clustering
Generative Models



Reinforcement Learning

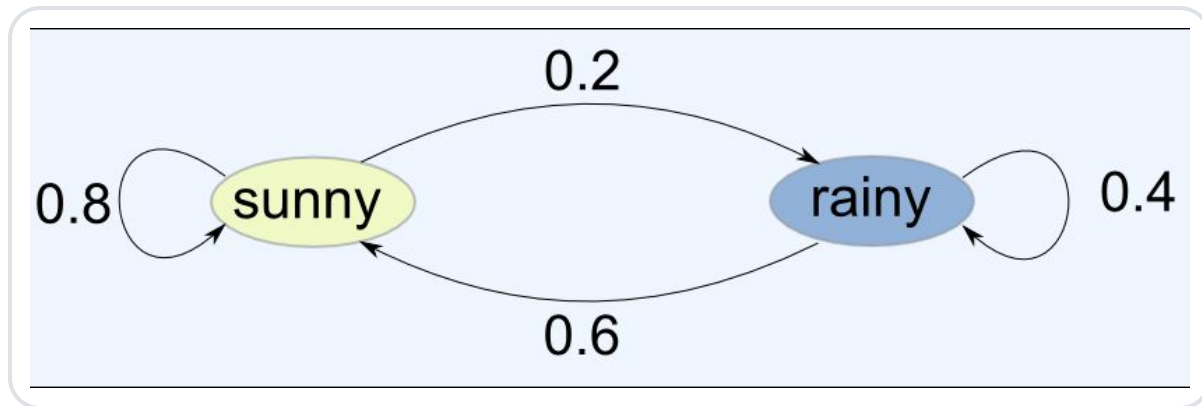
- Agent interacts with an environment and learns to maximize a reward
- Game Playing
Robot Navigation



Markov Process

A **Markov Process** is a stochastic model describing a sequence of possible events in which the probability of each event depends only on the state attained in the previous event.

State at time $t + 1$ solely depends on state at time t



Markov Decision Process (MDP)

Framework for modeling decision-making in situations where outcomes are partly random and partly under the control of a decision maker

 $\mathcal{S}$: $\mathcal{A}$: $\mathcal{P}$: $\mathcal{R}$: γ :

State Space

$$\mathcal{S} = \{s_1, s_2, s_3, \dots\}$$

- $\mathcal{S}$: Set of states representing the environment.

Captures everything the agent knows about the environment

E.g. position of enemies,
position of itself, walls
(can be complete or partial)



$\mathcal{S}$:

$\mathcal{A}$:

$\mathcal{P}$:

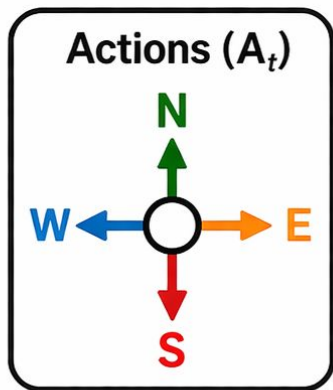
$\mathcal{R}$:

γ :

Action Space

$$\mathcal{A} = \{\text{Up, Down, Left, Right}\}$$

- $\mathcal{A}$: Set of actions the agent can take.



$\mathcal{S}$: State

$\mathcal{A}$: Action

$\mathcal{P}$:

$\mathcal{R}$:

γ :

Transition Function

- $\mathcal{P}$: Transition probability function, $\mathcal{P}(s'|s, a)$.

The agent's actions do not always go as planned:

E.g. wind / noise sends agent 80% of the time to their intended direction, 20% to a random direction



$\mathcal{S}$: State

$\mathcal{A}$: Action

$\mathcal{P}$: Transition

$\mathcal{R}$:

γ :

Transition Function

- $\mathcal{P}$: Transition probability function, $\mathcal{P}(s'|s, a)$.

1 2 3 4 5

- For Markov decision processes, “Markov” means:

$$P(S_{t+1} = s' | S_t = s_t, A_t = a_t, S_{t-1} = s_{t-1}, A_{t-1}, \dots, S_0 = s_0)$$

=

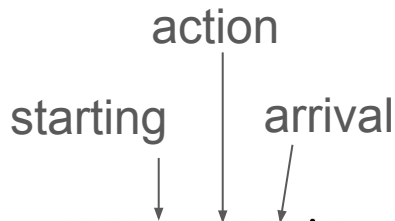
$$P(S_{t+1} = s' | S_t = s_t, A_t = a_t)$$

1



Reward Function

– $\mathcal{R}$: Reward function, $\mathcal{R}(s, a, s')$



The rewards the agent picks up through its actions:

E.g. +1 for gold coin, +100 for finishing the game, -100 for dying.

Most settings have small negative reward each step for time passing (or positive reward for staying alive)



$\mathcal{S}$: State

$\mathcal{A}$: Action

$\mathcal{P}$: Transition

$\mathcal{R}$: Reward

γ :

Discount Factor

- $\gamma \in [0, 1]$: Discount factor that balances immediate and future rewards.



now

vs.



later

How much do we discount a reward that is picked up next step vs now.

Typical values: $\gamma=0.9$

Q: What happens if $\gamma=1$ or $\gamma=0$?

$$V(s) = \sum_{t=0}^T \gamma^t R(s_t, a_t, s_{t+1})$$

Discount
factor

Reward given state,
action at t



$\mathcal{S}$: State

$\mathcal{A}$: Action

$\mathcal{P}$: Transition

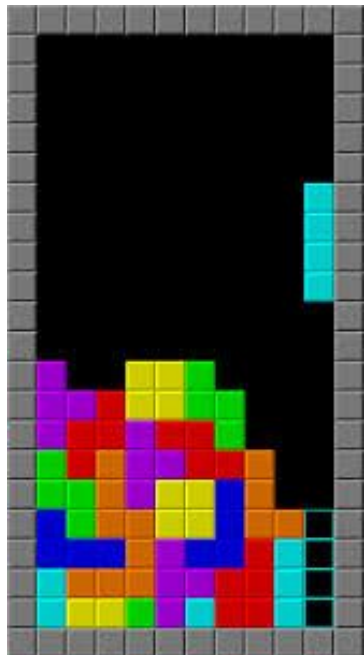
$\mathcal{R}$: Reward

γ : Discount Factor

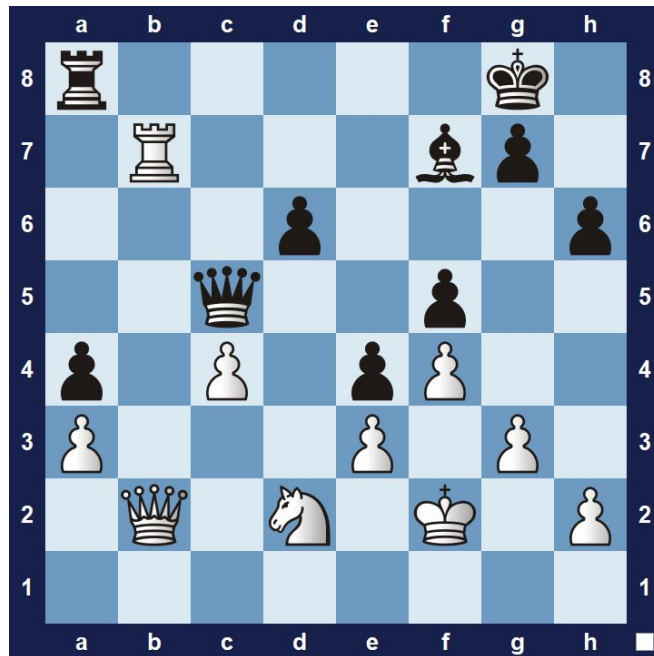
Markov Decision Process (MDP)

- MDPs provide a framework for modeling sequential decision-making problems.
- An MDP is defined by a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$:
 - $\mathcal{S}$: Set of states representing the environment.
 - $\mathcal{A}$: Set of actions the agent can take.
 - $\mathcal{P}$: Transition probability function, $\mathcal{P}(s'|s, a)$.
 - $\mathcal{R}$: Reward function, $\mathcal{R}(s, a, s')$
 - γ : Discount factor, $\gamma \in [0, 1]$.

Discussion: Identify the **state**, **action**, **transition** and **reward** in each.



Tetris



Chess



Upside-down helicopter flight

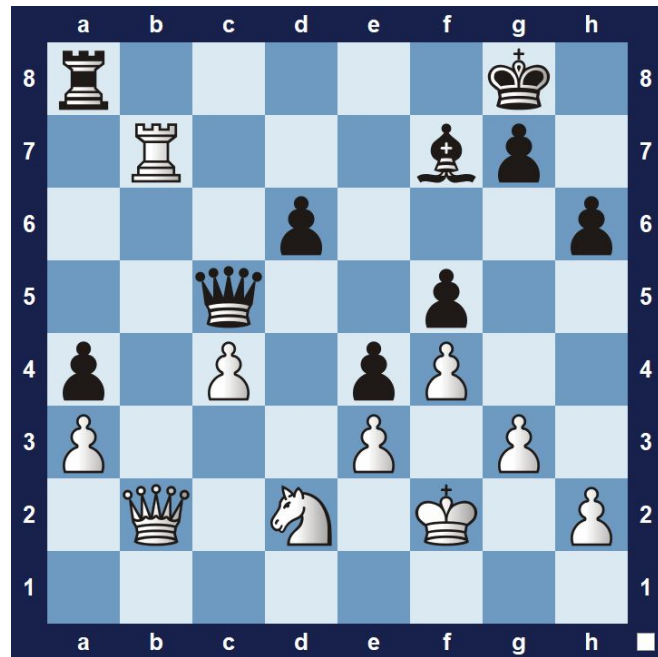
Markov Decision Process (MDP)

An MDP is defined by a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$

Framework for modeling agent in environment

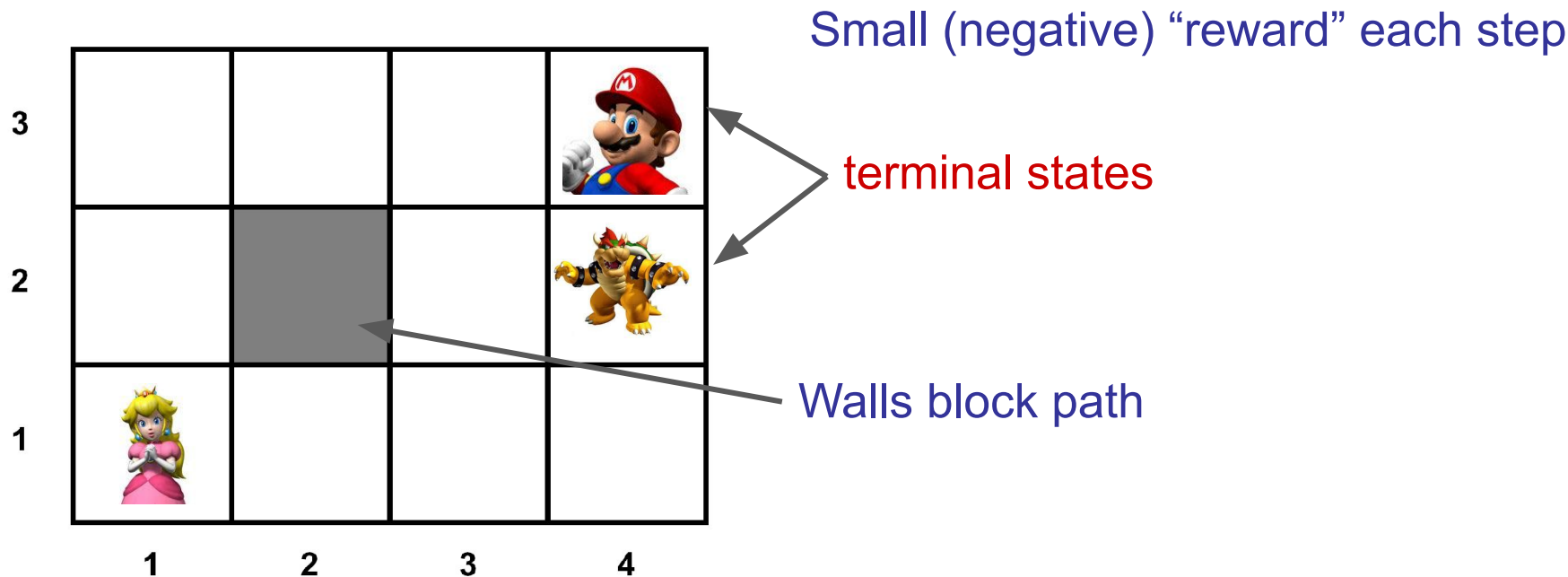
We still need to find agent **policy** to maximize reward!!!

E.g. which piece to move to max chance of winning



Simplified Setup

Princess Peach gets reward +1 for Mario, but -1 for Bowser



Policy

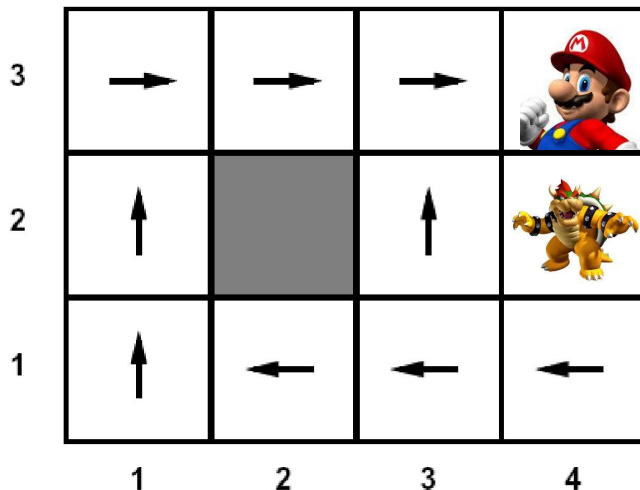
- In an MDP, we want an optimal policy $\pi^*: S \rightarrow A$
 - A policy π gives an action for each state
 - An optimal policy maximizes expected cumulative reward if followed

Optimal policy when
 $R(s, a, s') = -0.03$ for all
 non-terminals s

$$V(s) = \sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1})$$

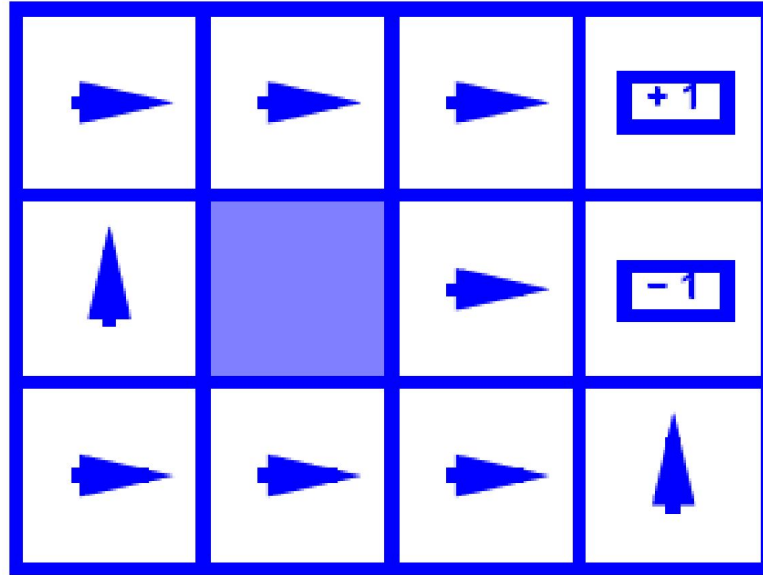
Discount
factor

Reward given state,
action at t



Are there any conditions where this policy is optimal?

$$R(s, a, s') = ??$$



Optimal Values

Why? Optimal values define optimal policies!

Value of a state s

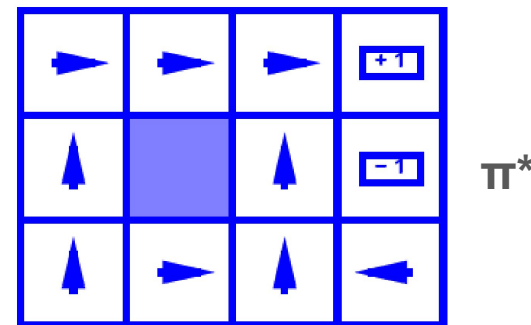
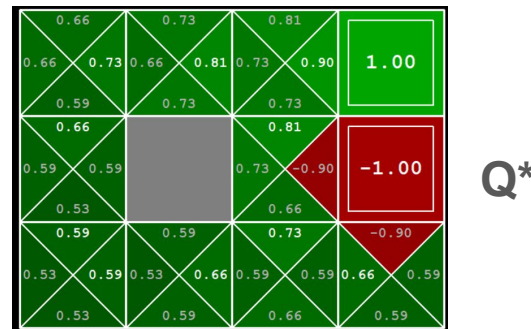
$V^*(s)$ = expected reward starting in s and acting optimally

Q-value of (s,a)

$Q^*(s,a)$ = expected reward starting in s , taking action a and thereafter acting optimally

Optimal policy

$\pi^*(s)$ = optimal action from state s



Bellman Equations

- Definition of “optimal values” leads to a simple one-step lookahead relationship

Optimal rewards = maximize over first action and then follow optimal policy

- Formally:

$$V^*(s) = \max_a Q^*(s, a)$$

$$Q^*(s, a) = \sum_{s'} \mathcal{P}(s'|s, a) [R(s, a, s') + \gamma V^*(s')]$$

$$V^*(s) = \max_a \sum_{s'} \mathcal{P}(s'|s, a) [R(s, a, s') + \gamma V^*(s')]$$

Value Iteration

- Idea:

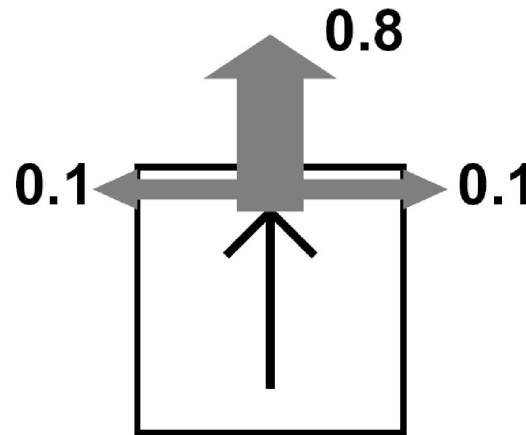
- Start with $V_0^*(s) = 0$
- Given V_i^* , calculate the values for all states at next step:

$$V_{i+1}(s) \leftarrow \max_a \sum_{s'} \mathcal{P}(s'|s, a) [R(s, a, s') + \gamma V_i(s')]$$

- This is called a **value update** or **Bellman update**
 - Repeat until convergence
- Theorem: will converge to unique optimal values
 - Basic idea: approximations get refined towards optimal values
 - Policy may converge long before values do

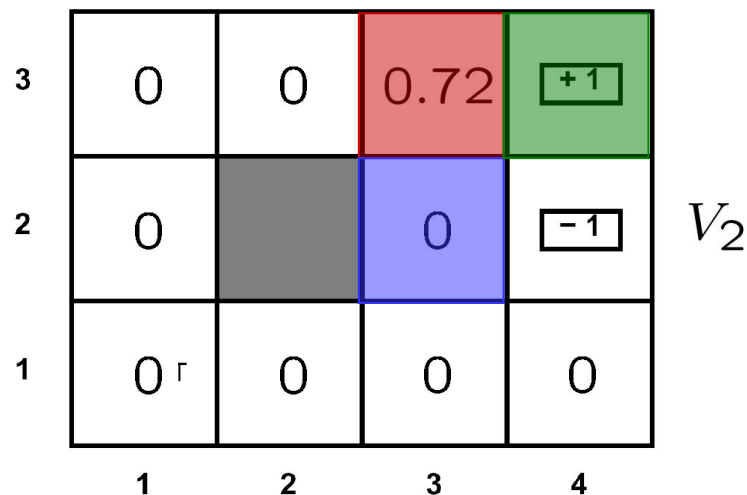
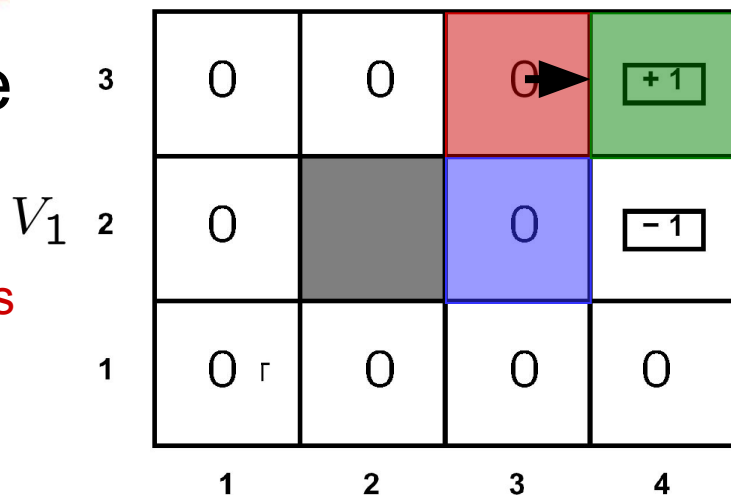
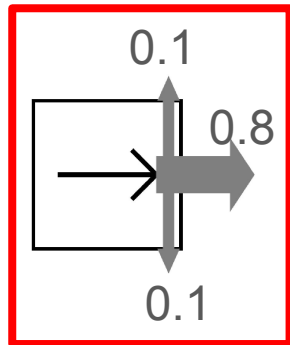
Noise Variable on $\mathcal{P}(s'|s, a)$

- 80% of the time, the action North takes the agent North
(if there is no wall there)
- 10% of the time, North takes the agent West;
10% East
- If there is a wall in the direction the agent would have been taken, the agent stays put



Example

Env Dynamics



$$V_{i+1}(s) = \max_a \sum_{s'} \mathcal{P}(s'|s, a) [R(s, a, s') + \gamma V_i(s')]$$

$$V_2(\langle 3, 3 \rangle) = \sum_{s'} \mathcal{P}(s' | \langle 3, 3 \rangle, \text{right}) [R(\langle 3, 3 \rangle) + 0.9 V_1(s')]$$

max happens
for a=right,
other actions
not shown

$$= 0.9 [0.8 \cdot 1 + 0.1 \cdot 0 + 0.1 \cdot 0]$$

Gridworld



Computing Policy

- Which action should we choose from state s :
 - Given optimal values V ?

$$\pi^*(s) = \arg \max_a \sum_{s'} \underbrace{\mathcal{P}(s'|s, a)}_{\text{Transition}} [\underbrace{R(s, a, s')}_{\text{Immediate reward}} + \underbrace{\gamma V^*(s')}_{\text{Discounted future}}]$$

- Given optimal q-values Q ?

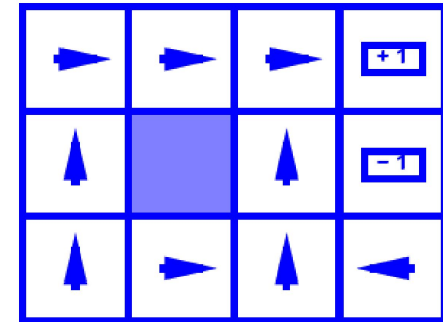
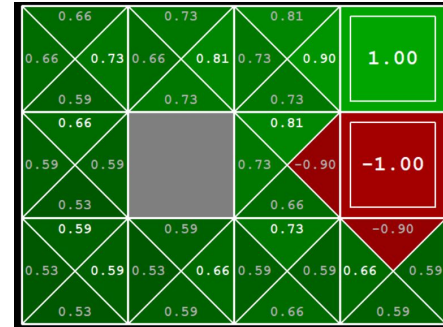
$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

- Lesson: actions are easier to select from Q 's!

Sub-optimal Values

- Imagine we don't have the optimal policy π^*
- We can define the value given a fixed policy:

$V^\pi(s)$ = expected reward starting in s and following π



Policy Iteration

- Policy evaluation: with fixed current policy π , find values with simplified Bellman updates:
 - Iterate until values converge

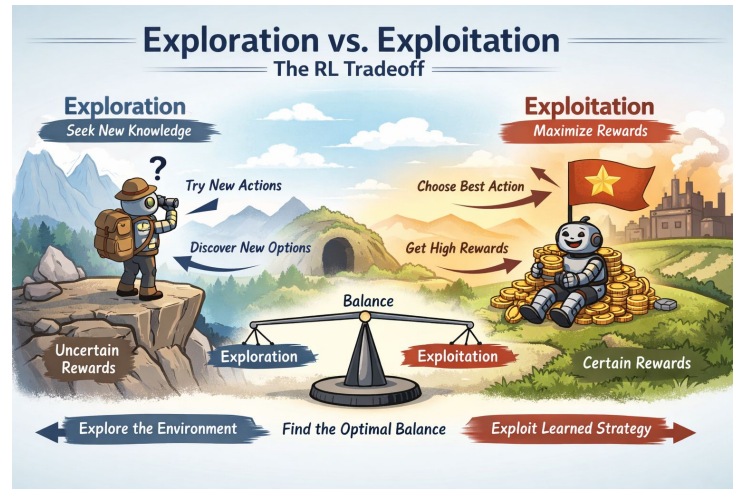
$$V_{i+1}^{\pi_k}(s) \leftarrow \sum_{s'} T(s, \pi_k(s), s') [R(s, \pi_k(s), s') + \gamma V_i^{\pi_k}(s')]$$

- Policy improvement: with fixed values, find the best action according to one-step look-ahead

$$\pi_{k+1}(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^{\pi_k}(s')]$$

Exploration-Exploitation TradeOff

- Problem: If policy is sub-optimal, you might never visit good states
- Solution ϵ -Exploration:
 - ϵ of the time, take a random action and don't follow policy
 - This gives you some non-zero probability of visiting every step
 - Slowly lower ϵ over time

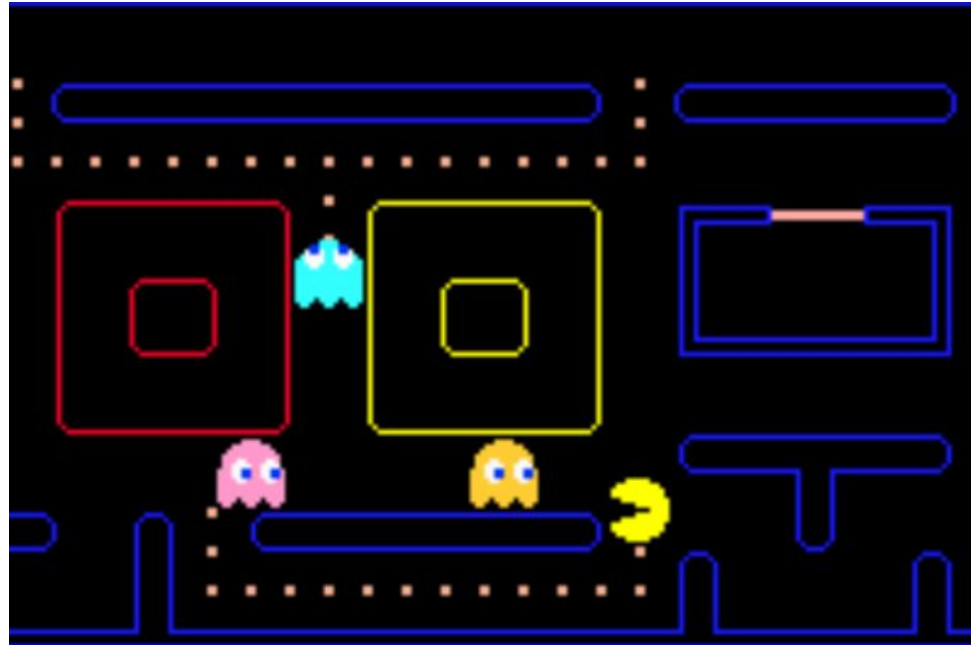


Calculate the size of the State Space for Pacman

in the following environment

Assume there are 100
positions, 4 ghosts, and
pacman

(Can ignore pellets for
simplicity)

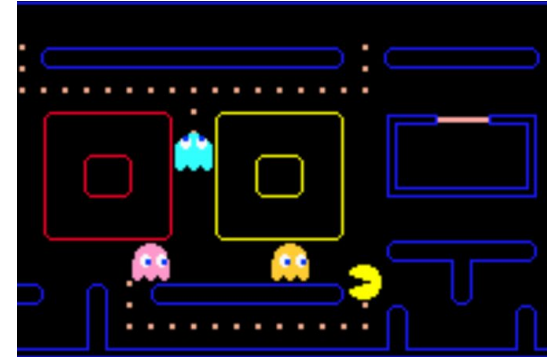


Calculate the size of the State Space for Pacman in the following environment

4 ghosts and Pacman: $100 \cdot 100 \cdot 100 \cdot 100 \cdot 100 = 10^{10}$

What if we consider pellets? Assuming there are 100 pellets:

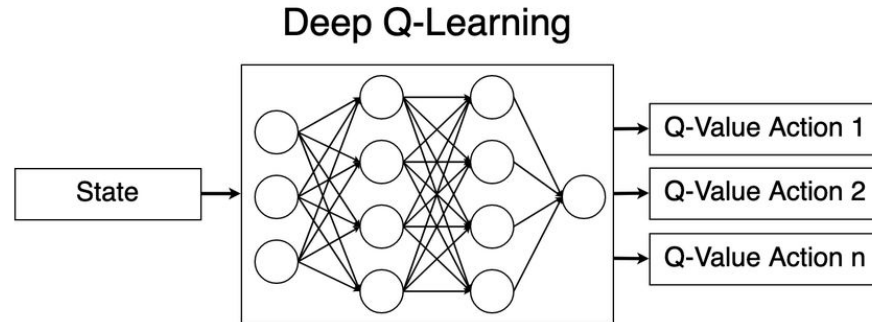
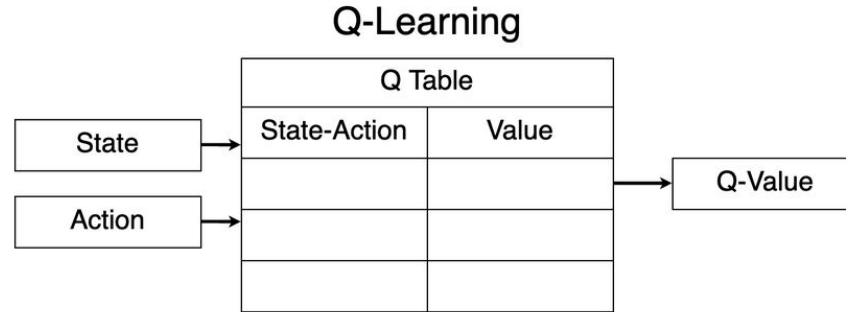
$$2^{100} \cdot 10^{10} > 10^{40}$$



Issue:

- As the state/action space increases, the likelihood of reaching a specific state and action decreases
- Unknown environment dynamics
- Sparse Rewards (agent doesn't know what is good or bad)

Next Time: How can we use deep learning to improve this?



Recap

- Markov Decision Processes (MDPs)
 - Framework for modeling decision-making in situations where outcomes are partly random and partly under the control of a decision maker
- Value function
 - Expected cumulative reward from state s following some policy
- Action-value (Q) function
 - Expected cumulative reward if you take action a in state s and follow some policy

Dense Rewards



Sparse Rewards

